

INF01085 - Sistemas Distribuídos E Tolerantes A Falhas – Turma U

PROF. WEVERTON CORDEIRO

SEMESTRE 2026/1

TRABALHO PRÁTICO PARTE 1: COMUNICAÇÃO CONFIÁVEL INTER-PROCESSOS

ESPECIFICAÇÃO DO TRABALHO

A proposta de trabalho prático é implementar um serviço distribuído de soma de números inteiros recebidos de clientes diversos. O objetivo do serviço é receber requisições de clientes diversos contendo um número a ser somado, processar cada requisição e somar o número recebido a uma variável acumuladora mantida pelo servidor, e exibir o resultado da soma após cada requisição recebida. Cada requisição conterá um número inteiro positivo, que será lido pelo cliente a partir da entrada padrão. O servidor deverá somar os números de todas as requisições recebidas de todos os clientes.

O principal critério de avaliação será a soma correta de todos os números enviados por todos os clientes nas requisições. Um critério secundário de avaliação será o tempo de resposta para o servidor receber, processar e responder um grande volume de requisições (10.000.000+ requisições) recebidas de clientes diversos.

A proposta deverá ser desenvolvida em duas etapas. A primeira etapa compreenderá funcionalidades que dependerão de tópicos como comunicação confiável inter-processos e sincronização para serem implementadas. O projeto deverá executar em ambientes Linux, nos computadores dos laboratórios de ensino do INF-UFRGS, mesmo que tenha sido desenvolvido em outras plataformas/outros computadores. O projeto deverá ser implementado em Python 3, usando exclusivamente a API User Datagram Protocol (UDP) para comunicação inter-processos.

DESCRIÇÃO DO SERVIÇO E FUNCIONALIDADES BÁSICAS

O projeto compreenderá dois programas distintos, sendo um cliente e um servidor. O servidor executará em uma única máquina, e receberá requisições de clientes diversos. Os clientes executarão em múltiplas estações (um cliente por estação).

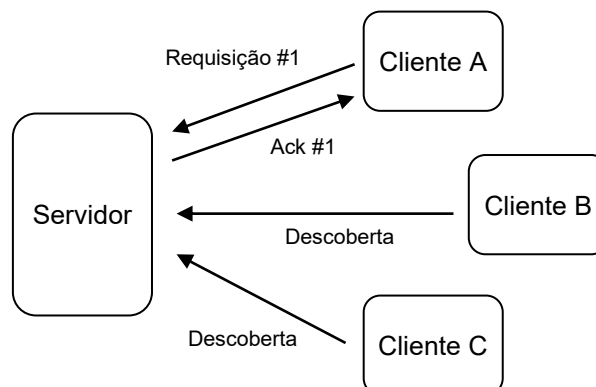


Figura 1. Dinâmica de funcionamento do serviço distribuído de agregação de dados.

A Figura 1 ilustra a dinâmica de funcionamento do serviço, que compreende duas fases: **descoberta** e **processamento**. Na fase de descoberta, o cliente recém inicializado envia na rede uma mensagem do tipo

DESCOBERTA (em modo de difusão, ou *broadcast*), para “descobrir” em qual endereço o servidor está escutando por requisições. O servidor, ao receber uma mensagem de descoberta 1) responde ao cliente (em modo ponto-a-ponto, ou *unicast*), confirmando assim em qual endereço está escutando por requisições e 2) registra o novo cliente em uma tabela de participantes.

Finalizada a fase de descoberta, o cliente passa para a fase de processamento. Nesta fase, o cliente envia um conjunto de mensagens do tipo *REQUISIÇÃO*, uma por vez. A requisição conterá o identificador da mensagem e um número inteiro. O servidor recebe cada mensagem, faz o processamento (soma o número contigo na mensagem de requisição a uma variável acumuladora mantida pelo servidor), e envia uma resposta (*acknowledgement*, ou *ack*). A resposta enviada pelo servidor confirma que a mensagem de requisição foi recebida e processada, e informa qual o resultado parcial da soma após o processamento desta requisição. O cliente somente prossegue com o envio da próxima requisição ao receber uma confirmação de que a requisição anterior foi processada. O cliente deverá aguardar um tempo e, caso não receba uma confirmação, deverá reenviar a requisição.

Sugere-se fortemente à equipe organizar cada subserviço do sistema em módulos para implementá-las, sobretudo para facilitar o processo de desenvolvimento da Etapa 2 do Trabalho Prático.

- **Subserviço de Descoberta**, para identificar em qual endereço IPv4 (Internet Protocol v4) o servidor está esperando por requisições. A equipe pode assumir que o servidor estará executando no mesmo segmento físico e lógico de rede. Assim, sugere-se usar *sockets* UDP configurados com a opção *broadcast* para implementar as rotinas de descoberta. Na primeira parte do trabalho prático, este subserviço (i) operará de forma passiva (isto é, recebendo e respondendo mensagens em *broadcast* do tipo *DESCOBERTA*) no servidor e (ii) operará de forma ativa (isto é, enviando mensagens de descoberta) nos clientes;
- **Subserviço de Processamento**, para receber as requisições dos clientes (contendo o número inteiro a ser somado), processá-las (somar o número inteiro recebido à variável acumuladora mantida no servidor) e enviar as respostas (confirmar que a requisição foi recebida, processada, e o resultado parcial da soma). Este subserviço (i) operará de forma ativa (isto é, enviando mensagens de requisições) nos clientes e (ii) operará de forma passiva (isto é, recebendo e respondendo requisições) no servidor;
- **Subserviço de Interface**, para exibir as mensagens de envio (e reenvio) de requisições e recebimento das respostas (no cliente) e para exibir as mensagens de recebimento e resultado do processamento das requisições (no servidor), além de acusar mensagens recebidas fora de ordem/duplicadas.

O serviço deve garantir, ainda:

- **Consistência nas estruturas de armazenamento:** As estruturas de armazenamento de dados no serviço devem ser mantidas em um estado consistente. Por exemplo, as informações sobre o resultado do processamento de todas as requisições devem ser mantidas de forma consistente.
- **Persistência de dados no servidor:** As estruturas de armazenamento de dados no serviço devem ser mantidas somente em memória. Na segunda parte do trabalho, mecanismos de replicação serão usados para garantir a persistência consistente das estruturas em múltiplas estações servidoras.

SOBRE AS REQUISIÇÕES

Para que o serviço de soma seja robusto contra falhas de rede, o protocolo de aplicação sobre UDP deve garantir que cada número enviado por um cliente seja somado ao acumulador global exatamente uma vez, independentemente de perdas de pacotes, duplicatas ou reordenação na rede. Cada estação cliente deverá ler cada requisição diretamente da entrada padrão (**teclado**), uma por vez, para enviá-las ao servidor. A Cada requisição é formada por um número identificador da requisição (que será incrementado em um a cada nova requisição enviada) e outro número inteiro (que corresponde ao número que deverá ser somado pelo servidor).

O primeiro número identificador de requisição que deverá ser usado por cada cliente é sempre 1. Este número deve ser incrementado em 1 a cada nova requisição enviada pelo cliente.

O servidor receberá a mensagem de requisição de um dado cliente e confirmará que o número de identificação recebido na mensagem é o próximo esperado. Caso positivo (o número de identificação recebido na mensagem é o

próximo esperado), o servidor atualizará o último número de requisição recebido, processará a requisição, e responderá ao cliente. Caso negativo, o servidor deverá responder a requisição com uma mensagem de ACK com o último número de identificação de requisição recebida e processada (dado que se tratará de uma retransmissão de uma mensagem para a qual o cliente não recebeu um ack do servidor).

Por outro lado, caso o servidor receba uma mensagem do cliente com um número de identificação superior ao próximo identificador esperado, o servidor deverá responder a requisição com uma mensagem de ACK com o último número de identificação de requisição recebida e processada, indicando assim que alguma requisição anterior foi perdida.

Caso o cliente não receba uma resposta do servidor até um tempo limite, o cliente deverá assumir que a mensagem foi perdida e deve reenviar a mensagem. Nenhuma nova mensagem de requisição deve ser enviada até o recebimento da confirmação da última requisição enviada. A equipe poderá assumir um valor de timeout de três vezes o RTT da rede local (medido com o comando `ping`), para melhorar o tempo de resposta. Se preferir, a equipe poderá usar o valor de timeout de 10 milissegundos.

IMPLEMENTAÇÃO DO SISTEMA

A proposta de trabalho prático está dividida em duas etapas, sendo que a segunda etapa irá contemplar funcionalidades adicionais. Portanto, considere uma implementação modular e com possibilidade de extensão, e o encapsulamento das funções de comunicação do cliente e do servidor em módulos isolados.

Para o funcionamento do serviço, o servidor deverá manter uma tabela de clientes participantes. Cada cliente é caracterizado pelo endereço IP (campo *address*), o número identificador da última requisição recebida e processada (campo *last_req*), e o valor da soma enviado ao cliente em resposta à última requisição processada (campo *last_sum*). Ao receber uma mensagem de descoberta de um novo cliente (via subserviço de descoberta), uma nova entrada deverá ser incluída na lista, com o os campos *last_req* e *last_sum* zerados. A Tabela 1 ilustra um exemplo dessa estrutura de dados.

Tabela 1. Exemplo de estrutura de dados de clientes mantida pelo servidor. Para manter a soma de uma grande quantidade de números, o projeto deve obrigatoriamente usar o tipo *unsigned int de 64 bits*

Endereço IP (<i>address</i>)	Última requisição recebida (<i>last_req</i>)	Última quantidade de requisições processadas enviado ao cliente (<i>last_num_reqs</i>)	Último valor da soma acumulada enviado ao cliente (<i>last_total_sum</i>)
1.1.1.1	0	0	0
1.1.1.2	2	2	43
1.1.1.3	6	8	111

No exemplo da Tabela 1, considera-se que o cliente 1.1.1.1 acabou de ingressar no sistema; o cliente 1.1.1.2, por sua vez, enviou duas requisições, a primeira contendo o número 9 e a segunda contendo o número 34; por fim, o cliente 1.1.1.3 enviou seis requisições (todas estas após o cliente 1.1.1.2), com os seguintes valores em cada requisição: 7, 1, 3, 19, 33, 5.

O servidor deverá manter ainda uma tabela para guardar informações sobre a soma agregada. Em resumo, o servidor deverá guardar a quantidade de requisições recebidas de todos os clientes (campo *num_reqs*) e o valor total da soma acumulada (campo *total_sum*).

Tabela 2. Exemplo de estrutura de dados da soma agregada mantida pelo servidor. Para manter a soma de uma grande quantidade de números, o projeto deve obrigatoriamente usar o tipo *unsigned int de 64 bits*.

Quantidade de requisições recebidas (<i>num_reqs</i>)	Valor total da soma acumulada (<i>total_sum</i>)
8	111

No exemplo da Tabela 2, registra-se que o servidor recebeu 8 requisições de todos os clientes, e o valor total da soma acumulada até o momento é 111.

Note que os valores de somas parciais serão não-determinísticos, e dependerão essencialmente da ordem em que as requisições forem recebidas no servidor e se haverá perda de pacotes e retransmissões no sistema.

INTERFACE DO USUÁRIO

SERVIDOR:

O servidor deverá ser OBRIGATORIAMENTE iniciado via linha de comando de forma simples, com a porta UDP a ser usada para comunicação inter-processos sendo recebida como parâmetro:

```
$ python3 servidor.py 4000
```

Após iniciar o servidor, este deverá OBRIGATORIAMENTE exibir uma mensagem na tela informando a data e hora atual, o número de requisições recebidas (0) e o valor da soma agregada (0), NO EXATO FORMATO ABAIXO:

```
2024-10-01 18:37:00 num_reqs 0 total_sum 0
```

A cada nova requisição recebida, o servidor deverá OBRIGATORIAMENTE exibir na tela a data e hora atual, o endereço IP de origem da requisição, o identificador da requisição recebida, o número total de requisições recebidas de todos os clientes (já incluindo a requisição atual) e a soma total acumulada (já incluindo o número contido na requisição atual). Por exemplo:

- Assumindo que o servidor acabou de receber a Requisição #1 do cliente 1.1.1.2 com o número 10

```
2024-10-01 18:37:01 client 1.1.1.2 id_req 1 value 10 num_reqs 1 total_sum 10
```

- Assumindo que o servidor acabou de receber a Requisição #1 do cliente 1.1.1.3 com o número 8

```
2024-10-01 18:37:02 client 1.1.1.3 id_req 1 value 8 num_reqs 2 total_sum 18
```

- Assumindo que o servidor acabou de receber a Requisição #2 do cliente 1.1.1.2 com o número 3

```
2024-10-01 18:37:03 client 1.1.1.2 id_req 2 value 3 num_reqs 3 total_sum 21
```

Em caso de requisição recebida em duplicidade de um determinado cliente, o servidor deverá reexibir a mensagem exibida ao receber a última mensagem do cliente, por exemplo:

- Assumindo que o servidor acabou de receber a Requisição #1 do cliente 1.1.1.3 com o número 8, sendo que a tabela de clientes indica que a última requisição recebida de 1.1.1.3 foi #1:

```
2024-10-01 18:37:02 client 1.1.1.3 DUP!! id_req 1 value 8 num_reqs 2 total_sum 18
```

CLIENTE:

O cliente também deverá ser OBRIGATORIAMENTE iniciado via linha de comando de forma simples, com a porta UDP a ser usada para comunicação inter-processos sendo recebida como parâmetro:

```
$ python3 cliente.py 4000
```

Ao iniciar, o cliente deverá OBRIGATORIAMENTE exibir na tela uma mensagem informando a data e hora atual e o endereço IP do servidor (após a descoberta ser finalizada):

```
2024-10-01 18:37:00 server_addr 1.1.1.20
```

A partir de então, o cliente deverá OBRIGATORIAMENTE ler da entrada padrão (teclado) o próximo número a ser somado, para ser enviado ao servidor. Importante: o cliente não deve exibir nenhuma mensagem na tela solicitando o próximo número a ser lido. O usuário deverá simplesmente digitar um número e apertar ENTER.

Assim, o cliente deverá criar uma mensagem de requisição e enviar ao servidor. Após receber cada resposta, o cliente deverá exibir uma mensagem na tela. Por exemplo:

- Assumindo que o cliente mandou a Requisição #1 do cliente 1.1.1.2 com o número 10

```
2024-10-01 18:37:01 server 1.1.1.20 id_req 1 value 10 num_reqs 1 total_sum 10
```

Caso o cliente não receba uma resposta a uma requisição e estoure o tempo de timeout

A interface do cliente deve ter uma *thread* para escrever as mensagens na tela, e outra thread para ler os comandos digitados pelo(a) usuário(a). Ao apertar CTRL+C (interrupção) ou CTRL+D (fim de arquivo), o processo cliente deverá encerrar, sinalizando ao *manager* que o(a) usuário(a) está saindo do serviço (similar a EXIT).

DESCRIÇÃO DO RELATÓRIO

A equipe deverá produzir um relatório fornecendo os seguintes dados:

- Explicação e respectivas justificativas a respeito de:
 - (A) Como foi implementado cada subserviço;
 - (B) Em quais áreas do código foi necessário garantir sincronização no acesso a dados;
 - (C) Descrição das principais estruturas e funções que a equipe implementou;
 - (D) Explicar o uso das primitivas de comunicação inter-processos usadas;
- Descrição dos problemas que a equipe encontrou durante a implementação e como estes foram resolvidos (ou não).

A **nota será atribuída baseando-se nos seguintes critérios**: (1) qualidade do relatório produzido conforme os itens acima, (2) correta implementação das funcionalidades requisitadas, (3) qualidade do programa em si (incluindo a implementação da interface exatamente como solicitado na especificação do trabalho, documentação do código, funcionalidades adicionais implementadas, etc.) e (4) qualidade da apresentação (o que inclui domínio sobre o trabalho realizado por cada integrante da equipe).

MÉTODOS DE AVALIAÇÃO

O trabalho deve ser feito em grupos de **QUATRO INTEGRANTES**. As pessoas participantes da equipe devem estar claramente identificadas no relatório e na apresentação. A avaliação do trabalho será pela análise da implementação, do relatório produzido e da apresentação e demonstração, incluindo o funcionamento correto e completo do sistema. A ausência de um(a) integrante da equipe no dia da apresentação implicará em conceito zero para a pessoa ausente (salvo por motivos excepcionais como por ex. de saúde, que deverão ser registrados via junta médica da UFRGS).

A apresentação dos trabalhos e demonstração prática em laboratório dos sistemas implementados será realizada presencialmente, conforme o cronograma da disciplina. O funcionamento correto e esperado dos sistemas implementados é um critério importante de avaliação, conforme mencionado no início deste documento.

Faz parte do pacote de entrega os códigos-fonte da implementação, um tutorial de como executar os códigos e o relatório em um arquivo ZIP. A implementação deve estar funcional para demonstração durante a apresentação pela equipe, em laboratório.

DÚVIDAS, QUESTIONAMENTOS E SUGESTÕES

Dúvidas, questionamentos e sugestões podem ser enviados por e-mail ou Moodle.